

Refatoração de Testes e Detecção de Test Smells

Relatório do trabalho prático

Disciplina: Teste de Software

Trabalho: Refatoração de Testes e Detecção de Test Smells

Aluno: Rafael Ganascini de Moura

Matrícula: 858012

Repositório: <https://github.com/RafaelMouraG/test-smelly>

1. Análise de Smells

A suíte original (`test/userService.smelly.test.js`) passa em todos os testes executados, mas tem vários Test Smells. Três deles são descritos abaixo.

1.1 Lógica Condicional no Teste (Conditional Test Logic)

Onde: teste deve desativar usuários se eles não forem administradores.

O teste percorre uma lista com um usuário comum e um administrador usando um `for` e, dentro dele, um `if/else` decide qual `expect` vai rodar. Cada asserção só é executada em um dos caminhos.

Por que é um mau cheiro: um teste deve ter um único caminho de execução, previsível e linear. Com laços e condicionais, o teste passa a ter lógica própria, que também pode ter bugs. Quem lê precisa “executar o teste de cabeça” para saber o que está sendo verificado.

Risco: se a lista mudar (por exemplo, ficar vazia ou o `isAdmin` for alterado), algumas asserções deixam de rodar e o teste continua verde sem verificar nada. Quando ele falha, a mensagem não diz claramente qual dos dois cenários (usuário comum ou administrador) quebrou.

1.2 Tratamento de Exceção com `try/catch` (Silent Failure)

Onde: teste deve falhar ao criar usuário menor de idade.

O `expect` está dentro do bloco `catch`. Se `createUser` **não** lançar a exceção, o `catch` nunca roda, nenhuma asserção é executada e o teste passa.

Por que é um mau cheiro: o teste deveria provar que a exceção acontece, mas só verifica a mensagem *caso* ela aconteça. A ausência do comportamento esperado não é detectada.

Risco: se alguém remover a validação de idade de `UserService`, o teste continua passando e o bug chega à produção sem aviso. É o tipo de teste que deixa um “mutante” sobreviver no teste de mutação, como vimos no trabalho anterior.

1.3 Teste Ansioso (Eager Test)

Onde: teste deve criar e buscar um usuário corretamente.

O mesmo teste executa duas ações diferentes (`createUser` e `getUserById`) e verifica várias propriedades do resultado de cada uma.

Por que é um mau cheiro: um teste deve verificar um único comportamento. Ao misturar duas ações, o nome do teste fica genérico e a fronteira entre *Arrange*, *Act* e *Assert* desaparece (o próprio código usa os comentários “Act 1” e “Act 2”).

Risco: quando o teste falha, não fica claro se o problema está na criação ou na busca. Além disso, a primeira falha interrompe o teste e esconde as verificações seguintes, o que dificulta o diagnóstico.

1.4 Outros smells identificados

- **Teste Frágil (Fragile Test):** o teste do relatório compara a linha formatada exata (`ID: ...`, `Nome: Alice`, `Status: ativo\n`). Qualquer mudança de formatação, mesmo sem mudar o comportamento, quebra o teste.

- **Teste Desabilitado e Vazio:** `test.skip('deve retornar uma lista vazia...')` com apenas um `TODO`. Ele aparece no relatório do Jest, mas não testa nada.
- **Convidado Misterioso / Fixture Genérica:** o objeto `dadosUsuarioPadrao` é definido no topo do arquivo, longe dos testes que o usam, o que obriga o leitor a procurar a origem dos dados.

2. Processo de Refatoração

O teste escolhido foi o de desativação de usuários, por concentrar lógica condicional, laço e duas regras de negócio diferentes em um só bloco.

Antes (`userService.smelly.test.js`)

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

Depois (`userService.clean.test.js`)

```
describe('deactivateUser', () => {
  test('desativa um usuário comum e retorna true', () => {
    // Arrange
    const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);

    // Act
    const resultado = userService.deactivateUser(usuarioComum.id);

    // Assert
    expect(resultado).toBe(true);
    expect(userService.getUserById(usuarioComum.id).status).toBe('inativo');
  });

  test('não desativa um administrador e retorna false', () => {
    // Arrange
    const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

    // Act
    const resultado = userService.deactivateUser(usuarioAdmin.id);

    // Assert
    expect(resultado).toBe(false);
    expect(userService.getUserById(usuarioAdmin.id).status).toBe('ativo');
  });

  test('retorna false quando o usuário não existe', () => {
    // Act
    const resultado = userService.deactivateUser('id-inexistente');

    // Assert
    expect(resultado).toBe(false);
  });
});
```

Decisões tomadas

1. **Um teste por cenário.** O laço com `if/else` virou dois testes independentes, um para cada regra de negócio. Sem condicionais, toda asserção sempre é executada, o que elimina os erros `jest/no-conditional-expect`.
2. **Padrão AAA explícito.** Cada teste tem as seções *Arrange*, *Act* e *Assert* bem separadas, com uma única ação (`deactivateUser`) por teste.
3. **Nomes descritivos.** O nome de cada teste descreve o comportamento esperado (“não desativa um administrador e retorna `false`”). Se ele falhar, o relatório do Jest já indica qual regra quebrou.
4. **Asserção que faltava.** O teste original só conferia o retorno `false` para o administrador. A versão nova também verifica que o status dele continua ativo, o que pegaria um bug em que o admin é desativado mas a função retorna `false`.
5. **Caso de borda novo.** Foi adicionado o cenário de id inexistente, que não estava coberto.
6. **Dados locais.** Cada teste cria os próprios dados, sem depender de uma fixture global (eliminando o Convidado Misterioso).

Resumo das outras refatorações

Smell original	Correção aplicada no arquivo limpo
<code>try/catch</code> no teste do menor de idade	<code>expect(() => ...).toThrow('O usuário deve ser maior de idade. ')</code> , que falha se a exceção não for lançada
Eager Test (criar + buscar)	Testes separados para <code>createUser</code> (dados, id, ids únicos, <code>isAdmin</code> , <code>createdAt</code>) e <code>getUserById</code> (encontrado e <code>null</code>)
Teste frágil do relatório	Verifica se o relatório contém id, nome e status de cada usuário, sem depender da formatação exata
<code>test.skip</code> vazio	Implementado como “informa que não há usuários quando nenhum foi cadastrado”
Validações sem teste	Novos testes para idade exatamente 18 e para campos obrigatórios ausentes (<code>test.each</code> com nome, email e idade)

O arquivo final tem 19 testes organizados em blocos descreve por método do `UserService`.

3. Relatório da Ferramenta (ESLint)

O ESLint foi configurado com `eslint-plugin-jest` usando o `.eslintrc.json` do enunciado (regras de `eslint:recommended` e `plugin:jest/recommended`, com `no-disabled-tests`, `no-conditional-expect` e `no-identical-title` explicitadas). Foi usada a versão 8 do ESLint, porque as versões 9 e 10 adotaram o novo formato de configuração (*flat config*) e não leem `.eslintrc.json`.

Primeira execução (npx eslint .)

```
rafael-moura@LarpMachine:~/faculdade/6 Período/test-smelly$ npx eslint .

/home/rafael-moura/faculdade/6 Período/test-smelly/test/userService.smelly.test.js
 44:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 46:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 49:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 73:7  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 77:3  warning Tests should not be skipped          jest/no-disabled-tests
 77:3  warning Test has no assertions          jest/expect-expect

* 6 problems (4 errors, 2 warnings)
```

Comentário

A ferramenta encontrou de forma automática três dos smells identificados na análise manual:

- **Linhas 44, 46 e 49:** os expect dentro do if/else do teste de desativação (Lógica Condicional).
- **Linha 73:** o expect dentro do catch do teste do menor de idade (Silent Failure).
- **Linha 77:** o test.skip sem asserções, apontado por duas regras ao mesmo tempo (teste desabilitado e teste sem expect).

Por outro lado, o linter **não** detectou o Eager Test, o Teste Frágil do relatório nem o Convidado Misterioso. Esses smells dependem da intenção do teste e do significado dos dados, e não de um padrão sintático. Isso mostra que a análise estática complementa a revisão humana, mas não a substitui.

Validação final

Depois da refatoração:

- `npx eslint test/userService.clean.test.js` não reporta nenhum erro ou aviso.
- `npm test` executa as duas suítes com sucesso: **23 testes aprovados** e 1 pulado (o test.skip do arquivo original, que foi mantido sem alterações, como pede o enunciado).

4. Conclusão

Testes que passam não são necessariamente testes que protegem o código. A suíte original estava verde, mas o teste do menor de idade continuaria passando mesmo sem a validação, e o teste de desativação tinha asserções que podiam deixar de rodar sem ninguém perceber. Testes limpos, com um comportamento por teste, estrutura AAA e sem lógica, tornam as falhas óbvias e as mensagens de erro úteis, e servem como documentação viva das regras de negócio.

As ferramentas de análise estática como o ESLint com eslint-plugin-jest tornam essa qualidade sustentável: elas verificam automaticamente, a cada execução, padrões que seriam fáceis de deixar passar em uma revisão manual, e podem ser integradas ao CI para impedir que novos smells entrem no projeto. Como o linter só enxerga padrões sintáticos, o melhor resultado vem da combinação das duas coisas: a ferramenta cuida do que é mecânico, e a revisão humana avalia a intenção e o design dos testes.